

Microscopic Traffic Simulation

Special topics on model calibration

Alessandro Scalese

March 13, 2026

Contents

1. Introduction	4
2. Model Calibration	6
2.1. Task 1: Evaluating simulation replications	6
2.1.1. The white noise hypothesis	6
2.1.2. Evaluating the required replications number	6
2.2. Task 2: Exploration in Goodness of Fit functions and input space	7
2.2.1. GoF exploration	7
MAPE	7
RMSE	8
RMSN	8
GLS-based weighting schemes	8
2.2.2. Input space exploration	9
2.3. Task 3: Calibration using SPSA	9
2.3.1. The SPSA algorithm	10
Modifications to the standard SPSA	10
Hyperparameter tuning	10
2.3.2. Simulation configuration	11
2.3.3. Objective function definition	11
2.3.4. Calibration results	12
2.4. Task 4: Advanced calibration with PC-SPSA	12
2.4.1. PCA analysis	12
2.4.2. Algorithm adaptation and tuning	13
2.4.3. Calibration results and comparison with plain SPSA	13
3. Dynamic applications	14
3.1. Task 1: Dynamic Dwell Times	14
3.1.1. Implementation details	15
3.2. Task 2: Request Stops	15
3.2.1. Implementation details	15
3.3. Task 3: Public Transport Prioritization	16
3.3.1. The Krefelder Straße Junction	16
3.3.2. Prioritization logic overview	17
Detection step	17
Active Prioritization Logic	17
3.3.2.0.1. Active phase: major	17
3.3.2.0.2. Active phase: bus	18
Offset Recovery	18
3.3.3. Results and Performance Evaluation	18
Dwell times and travel time	19
Environmental impact	19
Effectiveness and trade-offs of Public Transport Prioritization	19
Comparison of active traffic control vs ddt and rs	20
Bibliography	21

To do:

- **p. 5:** REFINE AFTER WRITING
- **p. 7:** insert figure: proportion of sensors with stat signif res
- **p. 7:** insert figure: average counts for excluded sensors
- **p. 9:** inserire primo plot rmsn
- **p. 9:** inserire plot rmsn relativo
- **p. 11:** insert plot: hyperparameter optimization results
- **p. 12:** insert plot: calibration results
- **p. 12:** insert plot: calibration with individual —
- **p. 12:** insert weights
- **p. 12:** insert actual values!!!
- **p. 13:** inserire plot comparison spsa pcspas
- **p. 14:** insert ref to transit capacity and quality of service manual - formula aligns with empirical models etc etc
- **p. 16:** ref
- **p. 16:** insert figure
- **p. 17:** insert figure of the complete diagram
- **p. 17:** insert detection step diagram
- **p. 18:** insert major phase diagram
- **p. 18:** insert bus phase diagram
- **p. 18:** insert offset recovery diagram
- **p. 19:** insert avg dwell times
- **p. 19:** insert travel time distribution plot
- **p. 19:** insert fuel plot
- **p. 19:** insert emission plot base
- **p. 19:** insert time-space diagram for sq
- **p. 19:** insert zoom
- **p. 20:** insert time-space diagram for PTP
- **p. 20:** insert crossing/corridor trt
- **p. 20:** insert travel time distr with ptp
- **p. 20:** insert fuel / emissions ptp

1. Introduction

Traffic simulation models are invaluable tools in traffic planning and management. Traffic forecasts are one of the foremost information tools used by decision makers in transportation matters such as transportation policies and infrastructure investments, with significant impacts on communities and society as a whole.

In the early 2000s, (Flyvbjerg et al., 2003; 2005) extensively analyzed the economic and traffic impacts of infrastructure projects undertaken in the previous decades. Among their findings, they highlighted that more than 50% of all projects reported differences in the forecasted and actual traffic demand above 20%, and that over 90% of the projects incurred in some measure of cost escalation, meaning the final cost was higher than the initial estimates. To top this off, they also underline that these — do not demonstrate — trends in their analysis period (ranging from the 1930s to the end of the 20th century).

This alone already does not paint a favourable landscape: inaccurate models can lead to possibly wrong or at least not optimal choices in transportation policies and infrastructure design, whose cost is more often than not underestimated, leading to misallocations of public money.

One of the reasons behind the inaccuracy of traffic forecasts is the complexity of traffic modeling itself: a complete traffic model must account for several interacting components, from the microscopic variables that influence driving behavior, to the drivers of route choice and pathing, all the way up to the definition of the overarching traffic flows. Each of these sub-components can be modelled in different ways, with different complexities and parameters, and each layer introduces errors and indirections which contribute to the inaccuracy of the final model. Furthermore, the lack of accurate data presents another challenge: the OD trips, which define the flows across the network zones, are not known a priori, but either have to be modeled from socio-economic and topological variables (as done in the four-step model for traffic assignment (Dios Ortúzar & Willumsen, 2024)) or estimated from traffic measurements (data-driven approach). These measurements are taken from traffic sensors such as induction loops or cameras, and their coverage of city networks is often sparse and uneven, leading to an undetermined optimization problem.

In this context, calibration can then be defined as the systematic adjustment of the model's parameters to minimize the discrepancy between real world measurements and the model's predictions. In transport modeling, these parameters are generally categorized into two groups: supply and demand. Supply parameters describe the physical and operational characteristics of the transportation system, including network geometry, speed limits, signal timings, as well as the microscopic driving behavior parameters that dictate how vehicles interact with the infrastructure (e.g. the lane-changing logic parameters). Demand parameters instead define the general population behavior - i.e. the location and amount of trips, which are typically represented in the OD matrix.

In our case, we are going to use a microscopic traffic simulator, SUMO (Lopez et al., 2018), as our model, using its default driving behavior and transport supply parameters. This leaves the demand parameters - the OD matrix - to be calibrated.

As the simulator acts as a “black-box” function, meaning we can only control the inputs (the OD matrix) and observe the outputs (the sensors measurements), but we do not have an analytical form that describes the relationship between them, we are precluded from using common gradient descent optimization algorithms due to computational limitations. In fact, estimating the gradient using (for example) a Finite Differences approach would require us to compute $2N$ function evaluations (i.e. simulations), where N is the number of parameters, which quickly becomes infeasible as the network grows (as both the number of parameters and the computational requirements of the simulations typically grow with the network’s size). Therefore, in transport model calibration, the optimization algorithm choice often falls onto the Simultaneous Perturbation Stochastic Approximation algorithm, originally developed by (Spall, 1998), which only requires 2 function evaluations to estimate the local gradient.

This algorithm and one of its variants will be used in the first part of this study, which focuses on the calibration of a traffic model for the city of Aachen. The second part of the study focuses instead on some dynamic applications such as Public Transport control and prioritization.

Indeed, this also showcases one of the unique advantages of SUMO: the ability to use its TraCI (Traffic Control Interface) API for fine-grained control over the simulation state, which allows us to implement real-time measurements and adaptive control strategies.

To do: REFINE AFTER WRITING In the following sections, —: The following chapters are organized as follows: Section 2 describes the Aachen network and the statistical determination of simulation replications; Section 3 details the SPSA-based calibration methodology and results; finally, Section 4 presents the TraCI-based implementation of dynamic Public Transport strategies.

2. Model Calibration

We are tasked with calibrating the origin-demand matrix for the morning peak-hour period (8-9am) for a medium sized network from the city of Aachen, in Germany. The network has been divided in twenty-three TAZes (Traffic Assignment Zones), which define the origin and destination of all trips on the network. This subdivision results in a total of 529 calibrable parameters (OD pairs). The simulation data will be collected by 599 synthetic loop detectors (around 15% coverage), which record counts, speeds and densities on a variety of links across the network.

2.1. Task 1: Evaluating simulation replications

2.1.1. The white noise hypothesis

Our simulator of choice (SUMO with mesoscopic simulations enabled) is inherently stochastic (noisy): several components of the model, which are designed to replicate human driving behavior (lane-changing decision making, rerouting probability), introduce various levels of randomness which have to be accounted for when analysing the model's output. As a consequence of this stochasticity, the same input parameters (OD flows) can lead to different results, meaning a single simulation run is not representative of the average traffic state across the network. To account for this variability, we need to adopt a “white noise” hypothesis, meaning that we assume that the variability in simulation outputs (meaning the error between them and the true values) follows a normal distribution with zero mean and no systematic bias.

Mathematically, we can express this as:

$$Y_i = \bar{Y} + e \quad (1)$$

where:

Y_i are the outputs of a single simulation run

$\bar{Y}$ is the true expected value of the system

e_i represents the stochastic noise

Under the white noise assumption, e is assumed to be independently and identically distributed, justifying the use of the sample mean across multiple simulation replications as a reasonable estimator of the network state.

2.1.2. Evaluating the required replications number

In order to determine the number of samples required to get statistically significant outputs with a 95% confidence interval and a 10% tolerance for each link, we are going to use a collection of data from 100 previous simulation runs. We can compute this number by applying the formula:

$$n = \left[\frac{2t_{(\alpha/2, n-1)} \cdot \sigma}{W} \right]^2 \quad (2)$$

where:

n number of required simulations

t t-students statistic (for $\alpha/2$ significance, $n - 1$ DOF)

σ standard deviation

W width of tolerance interval

This computation needs to be performed for each sensor, across the range of simulations we want to investigate. A sensor is considered to give statistically significant results if the resulting required number of simulation replications, n , is lower than or equal to the number of simulations for which the test was conducted.

We decided to test the statistical significance of the sensors for up to 30 simulation runs to evaluate the trade-off between significance of the results and processing time. In **To do: insert figure: proportion of sensors with stat signif res**, it can be seen that, for 15 simulation runs, over 90% of the sensors provide statistically significant outputs. Further increasing the number of simulations does not significantly increase the percentage of compliant sensors, at least not in a way that justifies the increased computational effort.

Furthermore, we also looked into the characteristics of the outputs of the sensors that did not meet the statistical significance requirement for 15 simulation runs: as shown in **To do: insert figure: average counts for excluded sensors**, the majority of the sensors not yielding statistically significant results are characterized by extremely low traffic volumes, which are inherently more sensitive to stochastic fluctuations.

Therefore, we chose to set the number of simulation replications to 15, and to exclude non-compliant sensors from the subsequent calibration process, in order to focus on the ones with the highest signal-to-noise ratio.

2.2. Task 2: Exploration in Goodness of Fit functions and input space

2.2.1. GoF exploration

In section 2.1.2, we briefly established that low traffic volumes are more sensitive to stochastic noise compared to higher ones. A similar pattern needs to be taken into account during the choice of a proper Goodness of Fit (GoF) function. A GoF is a function that quantifies the “distance” of our model’s results from the true values. It is the main indicator on which the optimization process is based, as it is used to derive the objective function value and, consequently, the direction of the optimization step.

It follows that the choice of a suitable GoF function is of paramount importance to the overall success of the calibration process (Hellinga, 1998). Some common objective functions, like the Root Mean Square Error (RMSE) and the Mean Absolute Percentage Error, are widely used in the literature as GoF functions for a variety of optimization problems. However, they may not be well-suited for the application on traffic calibration problems.

MAPE

The definition of MAPE computes the percentage error for each measurement by dividing the error for the true value:

$$\text{MAPE} = \frac{1}{N} \sum_{i=1}^N \left| \bar{y}_i - \frac{y_{\text{sim},i}}{\bar{y}_i} \right| \quad (3)$$

where:

N number of observations (sensors)

$\bar{y}_i$ true value for sensor i

$y_{\text{sim},i}$ observed (simulated) value for sensor i

From the formula, it is evident that, for small flows, the value of the function can explode. This would result in a disproportionate importance given to secondary flows, which as we have seen are also the ones more sensitive to stochastic variability.

RMSE

The RMSE is defined as the square root of the mean square error:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\bar{y}_i - y_{\text{sim},i})^2} \quad (4)$$

where N , $\bar{y}_i$ and $y_{\text{sim},i}$ are defined as in the previous section.

As the RMSE squares the individual errors before averaging, it penalizes larger errors more heavily than smaller ones, therefore tending to be dominated by the errors on high-flow links (Toledo & Koutsopoulos, 2004). Moreover, it is an absolute metric, meaning that it weighs errors of different significance (e.g. an error of 100 vehicles on an Autobahn and on a residential street) on the same scale.

RMSN

The Normalized Root Mean Square Error (RMSN or NRMSE) is computed by weighing the RMSE against the global mean of the observations:

$$\text{RMSN} = \frac{\sqrt{N \cdot \sum_{i=1}^N (\bar{y}_i - y_{\text{sim},i})^2}}{\sum_{i=1}^N y_i} \quad (5)$$

In contrast with the previous options, RMSN effectively balances the aggregate error by the total observed volume, avoiding both instability on small flows and the dominance of potential outliers. On top of this, it also is a dimensionless value, which also allows for a fair comparison across networks and varying traffic conditions. It is then a robust objective function choice for traffic calibration problem.

GLS-based weighting schemes

While the error metrics presented so far treat all observations with equal structural weights, a Generalized Least Squares (GLS) framework allows to explicitly incorporate the uncertainty associated with the data in the objective function formulation, effectively weighing the measurements based on their reliability. (Cascetta et al., 1993) introduced a simple weighting scheme that makes use of the observations' variance-covariance matrix:

$$\text{GLS} = (Y_{\text{sim}} - \bar{Y})^T V^{-1} (Y_{\text{sim}} - \bar{Y}) \quad (6)$$

where:

Y_{sim} observations vector

$\bar{Y}$ true values vector

V^{-1} inverse of the variance-covariance matrix

Using this approach, the residuals are weighed individually by the inverse of their variance, down-weighting less reliable observations such as measurements from sensors on low-flow links, which as stated before exhibit higher relative variance and are more susceptible to the simulator’s stochastic noise. In an optimization perspective, this ensures that the calibration process is guided by high quality data points (i.e. sensors with high signal-to-noise ratios), thus reducing the impact of the simulator’s stochasticity on the calibration process.

This aligns with what we have done in terms of filtering out non statistically significant measurements: our approach can thus be seen as a simplification of a GLS weighting scheme, as we are removing measurements with high intrinsic variance from the GoF computation.

2.2.2. Input space exploration

After establishing the use of the RMSN as the primary GoF metric, we conducted a line search to explore the sensitivity of the simulation to global variations in the input demand. We evaluated the performance of the model under different initial OD matrix multipliers, ranging from 0.1 to 2.0 with a 0.1 step size by computing our chosen GoF metric (RMSN) for three measures of performance (MOPs): counts, speeds, and densities.

The primary goal of this initial search is to identify potential systematic biases in the initial (a priori) OD matrices (i.e. a substantial over- or under-estimation of the real OD values). This way, we can select a starting point that is closer to the —.

The results of the exploration reveal distinct sensitivities for the three MOPs, as can be seen in

To do: inserire primo plot rmsn:

- For traffic counts, the minimum RMSN occurs at a multiplier of 0.8.
- For densities, the minimum RMSN occurs at a multiplier of 0.6.
- For speeds, the RMSN remained consistently low (around 10%) across the whole multiplier range, showing a distinctly low variability.

These initial results align with the findings of (Toledo & Koutsopoulos, 2004), who showed that different Measures of Performance for the same — may yield conflicting results.

To better compare the sensitivity of these metrics, given their different scales (especially speeds vs the others), we normalized the RMSN values by the minimum ones across the multiplier range. This allows us to analyze the relative — of the metrics, highlighting the MOPs that are most responsive to demand changes. As shown in **To do: inserire plot rmsn relativo**, OD changes in the explored range only have a marginal impact on the speeds, while the U-shaped counts and densities curves highlight their higher sensitivity to demand changes.

2.3. Task 3: Calibration using SPSA

Following the preliminary input space exploration, we implemented the SPSA algorithm (Spall, 1998) to perform the calibration of the Origin-Destination matrix. The algorithm implementation is based on a slightly modified version of the standard SPSA, with magnitude-dependent perturbation scaling and a common random numbers strategy for variance reduction. All relevant model and simulation setups are explained in the subsections below.

2.3.1. The SPSA algorithm

The SPSA algorithm updates the parameter vector θ (in our case, the flattened OD matrix) iteratively using a stochastic approximation of the gradient:

$$\theta_{k+1} = \theta_k - a_k \cdot \hat{g}_k(\theta_k) \quad (7)$$

The gradient $\hat{g}_k$ is estimated by perturbing all elements of θ_k simultaneously using a random vector Δ_k , drawn from a ± 1 Bernoulli distribution:

$$\hat{g}_k(\theta_k) = \frac{y(\theta_k + c_k \Delta_k) - (y(\theta_k - c_k \Delta_k))}{2c_k \Delta_k} \quad (8)$$

The algorithm's hyperparameters, a_k and c_k , control the step size and the perturbation magnitude, respectively, and are typically defined as follows:

$$a_k = \frac{a}{(k + 1 + A)^\alpha} \quad (9)$$

$$c_k = \frac{c}{(k + 1)^\gamma} \quad (10)$$

where a , c , A , α and γ are tunable hyperparameters that influence the convergence properties of the algorithm. The choice of these hyperparameters is crucial for the performance of the algorithm, as they affect the balance between exploration and exploitation in the optimization process.

Modifications to the standard SPSA

In our implementation, we introduced three modifications to the standard SPSA algorithm:

1. **Magnitude-dependent perturbation scaling:** instead of using a constant perturbation magnitude c_k for each element of the parameter vector, we scale it based on the magnitude of the parameters themselves, which helps maintain a consistent level of relative perturbation across parameters of different scales.
2. **Common random numbers strategy:** to reduce the impact of the simulator's stochasticity on the gradient estimation, we use common random numbers for both the positive and negative perturbations of the parameter vector. This should help isolate the effect of the perturbations from the inherent noise in the simulation, potentially leading to faster convergence. Empirically, we observed that the approach did indeed reduce the oscillations in the optimization process, especially in early algorithm iterations.
3. **Clipping of the parameter updates:** to ensure non-negative OD flows and to prevent excessively large updates of the parameter vector, we implemented a clipping mechanism that bounds the updates in a $\pm 15\%$ range of the current parameter values.

Hyperparameter tuning

As stated before, the choice of the SPSA hyperparameters is crucial for the convergence and performance of the algorithm. While there are some general guidelines for setting these hyperparameters, which can be found in the literature (Spall, 1998) and provide an acceptable starting point, their optimal values can vary significantly depending on the specific characteristics of the

problem. In our case, while maintaining the standard theoretically optimal values for the decay parameters α and γ (0.602 and 0.101, respectively) and for the stability constant A (around 10% of the total number of iterations), we performed an automatic search for the optimal values of the initial step size a and perturbation magnitude c .

More specifically, we leveraged the Optuna Python package (REFS) and its implementation of the Tree-structured Parzen Estimator (TPE) algorithm to perform a Bayesian optimization of the hyperparameters, allowing us to efficiently explore the hyperparameter space. After defining the search space for a (1, 150) and c (0.01, 1), we ran multiple optimization trials whose results can be seen in **To do: insert plot: hyperparameter optimization results**. For each trial, we ran the SPSA algorithm for a fixed number of iterations (20) and evaluated the resulting RMSN (based on counts) at each iteration, using Optuna’s pruning mechanism to stop unpromising trials early and focus computational resources on the most promising hyperparameter configurations.

The optimal values identified at the end of the optimization process were $a = 100$ and $c = 0.6$.

2.3.2. Simulation configuration

For the calibration process, each simulation evaluation simulated a one-hour period (8-9am), with a warm-up period of 30 minutes (7:30-8am) to allow the system to reach a steady state before collecting data for the GoF evaluation, and a cool-down period of 30 minutes (9-9:30am) to allow the system to dissipate any congestion caused by the high demand during the peak hour. This configuration allows us to ensure the system reaches a steady state before collecting sensor data.

Each simulation was replicated 15 times, and the entire calibration process was run for a total of 500 iterations.

Interestingly, while the initial input space exploration suggested an optimal global scale of 0.8, early tests revealed that initializing the SPSA with a multiplier of 0.9 led to faster and more stable convergence of the algorithm.

2.3.3. Objective function definition

As discussed in the previous sections, we chose the RMSN as our GoF metric for the calibration process. In particular, we tested various GoF definitions, which differ in the weights given to the different MoPs in the RMSN computation. This allows us to analyze the impact of the choice of the GoF definition on the calibration results. The general GoF metric was defined as the weighted sum of the three MOP’s RMSNs:

$$\text{GoF} = w_c \cdot \text{RMSN}_{\text{counts}} + w_d \cdot \text{RMSN}_{\text{densities}} + w_s \cdot \text{RMSN}_{\text{speeds}} \quad (11)$$

The weight combinations tested were the following:

- $w_c = 0.40, w_d = 0.50, w_s = 0.1$
- $w_c = 0.55, w_d = 0.35, w_s = 0.1$
- $w_c = 0.65, w_d = 0.25, w_s = 0.1$

These combinations were chosen based on the results of the line search on the initial OD matrix, which showed that the speeds were relatively insensitive to changes in the demand, while counts and densities were more responsive. We thus decided to give a low weight to the speeds RMSN,

while exploring different weight combinations for the counts and densities RMSNs to analyze their impact on the calibration results.

2.3.4. Calibration results

The results of the calibration process for the three experiments are shown in **To do: insert plot: calibration results**. All three weight combinations show a sharp initial RMSN descent in the early iterations, regardless of the specific weight combination, which suggests that the algorithm is able to quickly identify a direction of improvement in the parameter space. After this initial phase, the RMSN continues to decrease more gradually, with some oscillations, which is expected given the stochastic nature of the simulator and the optimization process.

What is more interesting is looking at the individual MOPs values across the iterations. In the figure **To do: insert plot: calibration with individual —** displays the decomposition of the unweighted RMSN components alongside the global metric for the **To do: insert weights** setup. It can be seen that, while the global weighted error stabilizes, the individual components continue to exhibit some — trends: the algorithm seems to keep trading off minor errors between counts and densities, as dictated by their assigned weights, while the speeds remain relatively unperturbed.

Overall, the final results demonstrate a successful calibration of our model for the Aachen network: **To do: insert actual values!!!**

2.4. Task 4: Advanced calibration with PC-SPSA

As an extension of our calibration approach, we then proceeded to implement the Principal Component SPSA (PC-SPSA) algorithm, proposed by (Qurashi et al., 2020). The main benefit of this algorithm is the reduction in the dimensionality of the optimization problem, achieved by performing Principal Component Analysis (PCA) on the historical data to identify the most significant directions of variability in the parameter space. This allows to focus the optimization process on the “directions” of maximum variability, which can lead to faster convergence and better results (Qurashi et al., 2020).

2.4.1. PCA analysis

In order to conduct the PCA analysis required to identify the principal components of the parameter space, we generated an initial population on the three available historical OD matrices, corresponding to different morning hours (7-8am, 8-9am, 9-10am). Following the methodology in (Qurashi et al., 2020), we generated 25 perturbations of each matrix by applying random multipliers to the OD flows:

$$OD_{\text{perturbed}} = (0.65 + 0.2 \cdot \delta) \cdot OD_{\text{base}} \quad (12)$$

where δ is a random variable drawn from a normal distribution with zero mean and standard deviation $\sigma = \frac{1}{3}$. This way, we generated a total of 75 perturbed OD matrices, which were then flattened and used as input for the PCA analysis.

The original parameter space consisted of approximately 520 non-null OD pairs. By applying PCA to the generated population, and selecting the principal components that cumulatively

explained 95% of the total variance, we ended up with $K = 29$ components, significantly reducing the dimensionality of the optimization problem.

2.4.2. Algorithm adaptation and tuning

In the PC-SPSA framework, the stochastic perturbations are not applied directly to the original parameter vector (i.e. to the elements of the matrix), but to the scores (eigenvalues) of the selected principal components. After the perturbation, the updated components are then projected back into the original space to reconstruct a matrix that can be used as input for the simulation.

As both the geometry and scale of the problem are significantly different from the previous direct approach, we had to implement some minor modifications to the SPSA algorithm:

- The magnitude-dependent perturbation scaling was adapted to the new problem space by using a relative (percentage) perturbation of the component scores
- The clipping was adapted to work on the scores, ensuring that in the minimization step, the component scores would not explode etc

Due to these changes and the new problem definition, the previously calibrated SPSA gain parameters a and c were found to be not applicable anymore. We thus conducted a new hyperparameter tuning phase via Optuna, which yielded the new PC-SPSA parameters:

- $a = 2$
- $c = 0.1$

Additionally, the stability parameter A was also adjusted and set to 20, following the common recommendation of setting it to around 10% of the total number of iterations, which in this case was set to 200 as we were expecting faster convergence compared to the plain SPSA.

All other simulation (warm-up and cool-down periods, number of replications) and algorithm aspects (common random numbers for plus/minus perturbations) were kept the same as in the previous calibration approach to ensure a fair comparison between the two methods.

2.4.3. Calibration results and comparison with plain SPSA

The implementation of the PC-SPSA variant yielded significant advantages compared to the plain SPSA approach, especially in regards of computational time and convergence speed. While the standard SPSA required hundreds of iterations to reach a stable solution, the PC-SPSA was able to achieve similar results in a fraction of the iterations without compromising the final calibration quality, leading to a significant reduction in the overall computational time required for the calibration process.

In the figure **To do: inserire plot comparison spsa pcspsa** we can see that the weighted RMSN achieved by PC-SPSA matches the results of the standard SPSA, but with a much faster convergence. This is likely due to the fact that the PC-SPSA focuses the optimization process on the most significant directions of variability in the parameter space, allowing it to more efficiently navigate towards optimal solutions. Moreover, the dimensionality reduction achieved by PCA also helps to mitigate the impact of the simulator's stochasticity on the optimization process, as it effectively filters out some of the noise in the parameter space.

3. Dynamic applications

We are now tasked with the implementation of real-time traffic control measures to improve the performance of a bus line in the Aachen network. In particular, we are to study the influence of dynamic dwell times and request stops on the bus line's performance, and to implement a traffic light prioritization strategy at a critical junction along the bus route.

The object of this study is the Bus Line 3 in Aachen. The line has a total of four stops: Hanse-mannplatz, Eurogress, Ehrenmal/Lousberg, and Ponttor. In the baseline scenario, the line runs on a fixed five-minutes schedule, with a fixed dwell time of 30 seconds at each stop, while the traffic lights along the route also operate on static, pre-timed signal plans.

In reality, dwell times can vary significantly between stops, due to varying number of people boarding or alighting the bus, and also due to the influence of random human behavior, making fixed dwell times a poor representation of the real-world operations. The implementation of dynamic dwell times should therefore lead to a more accurate representation of the bus line's performance.

Comparing different control strategies, such as dynamic dwell times and request stops, also allows us to compare their effectiveness in relation with other metrics such as line travel times, reliability, fuel consumption and emissions. This can provide help identify the most effective measures for improving the performance of the bus line while minimizing its environmental impact.

The implementation of these strategies is done via the TraCI python API for SUMO, allowing us to interact with the simulation in real time. In order to evaluate the impacts of each strategy, we will compare their results with the ones from a baseline "status quo" scenario, in which no real-time control measures are implemented. Each simulation will be replicated five times to account for the stochasticity of the simulator, and the results will be averaged across the replications to get a more accurate estimate of the strategies' performance.

The details of each strategy and their implementation are described in the respective sections. Their results and impacts are then discussed in the section 3.3.3.

3.1. Task 1: Dynamic Dwell Times

The first strategy requires the implementation of dynamic dwell times (DDT) at the bus stops. The dwell time t_{dwell} is calculated dynamically based on real-time passenger demand using the following formulation:

$$t_{\text{dwell}}(s) = 15 + 0.5 \cdot (n_{\text{board}} + n_{\text{alight}}) \quad (13)$$

where n_{board} and n_{alight} represent the number of passengers boarding and alighting the bus at the stop, respectively. **To do:** insert ref to transit capacity and quality of service manual - formula aligns with empirical models etc etc

3.1.1. Implementation details

The duration of the individual stops was set using a variety of TraCI's commands. As TraCI does not provide a direct way to check the number of passengers boarding and alighting the bus at each stop, we had to implement a custom logic to compute these values in real time.

For the alighting passengers, we used `traci.vehicle.getPersonIDList` to retrieve the list of current passengers for each bus, and then used `traci.person.getStage` to get the individual travel stages. The number of alighting passengers was then computed by counting the number of passengers whose destination stop matches the current (next) stop.

For the boarding passengers, we hold a list of the passengers before the vehicle stops in the cache, and check it against the list of passengers during the stop. This allows us to dynamically increase the dwell time if passengers appear at the bus stop after the bus has already stopped, compared to a static check at the start of the stop duration.

The stop duration itself is set using the `traci.vehicle.setStop` method.

3.2. Task 2: Request Stops

The second strategy requires the implementation of request stops (RS) on top of the dynamic dwell times. A request stop is a stop that the bus will only serve if there are passengers waiting to board or alight at that stop. If there are no passengers, the bus will skip the stop, leading to potential reductions in travel time and fuel consumption.

3.2.1. Implementation details

The implementation of request stops builds upon the dynamic dwell times logic, as we already have a way to compute the number of boarding and alighting passengers at each stop. The main difference is that we need a way to compute the number of boarding passengers before the bus reaches the stop, in order to decide whether to stop or not. Therefore, we need a method to check for the presence of passengers waiting at the stop, and a method to check the bus distance from the stop to decide when to perform this check.

For the bus distance from the stop, we use the `traci.vehicle.getDrivingDistance` method, which allows us to retrieve the distance of the bus from a specific point on the network. The exact edge and position of the bus stop are instead retrieved via the `traci.vehicle.getStops` method.

When this distance is lower than 60m, we then check the number of waiting persons at the bus stop with the `traci.busstop.getPersonCount` method, which allows us to retrieve the number of passengers waiting at a specific stop. If there are passengers waiting, the bus will proceed to stop as usual, while if there are no passengers, it will skip the stop and continue on its route.

The stop skipping is ensured by our use of the `traci.vehicle.setStop` method, which ensures the stop is canceled from the vehicle route. In earlier iterations, we tried to set the stop duration via `traci.vehicle.setStopParameter`, which instead led to inconsistent results (i.e. buses stopping for zero seconds).

The dwell time is then computed at the bus stop as in the previous strategy, allowing late passengers to board the bus and count towards the total dwell time.

3.3. Task 3: Public Transport Prioritization

The third strategy involves the implementation of a real-time Public Transport Prioritization (PTP) logic at a critical junction along the bus route (Krefelder Straße / Monheimsallee). The goal of PTP is to minimize the delay experienced by the bus at traffic signals, which is often one of the main sources of delay for urban transit. **To do: ref**

A naive approach to PTP would involve granting “absolute priority” to approaching buses. However, while absolute priority mechanisms significantly reduce transit delays, they also introduces cycle deviation, potentially leading to the disruption of overall traffic flow and coordinated signal timings.

To prevent traffic flow degradation, we are tasked to implement a strategy to avoid bus delays and compensate for the opposing traffic streams. Our implementation approach is composed of two primary modules: an Active Prioritization module and an Offset Recovery module. Each of them is described in detail in the following sections, after a brief overview of the junction and of the baseline scenario.

3.3.1. The Krefelder Straße Junction

An aerial view of the junction (as taken from the SUMO interface) can be seen in **To do: insert figure**. The object of our analysis is the junction in the top right of the figure, which is the one located on the bus route. The junction is a four-leg signalized intersection. In the baseline scenario, the traffic light operates on a fixed cycle, with three active phases (internal names used in the rest of the document in brackets):

- a phase serving the primary cross-traffic (Krefelder Straße) [major phase]
- a short left-turn phase for turning vehicles (Krefelder Straße to Ludwigsallee) [major-left phase]
- a bus phase for the transit corridor (Monheimsallee) [bus phase]

The length of the fixed signal plan is 90 seconds, with a fixed green time of 38 seconds for the major phase, 6 seconds for the major-left phase, and 37 seconds for the bus phase. The three active phases are separated by standard 3-second “interstage” (yellow/all-red) clearance intervals.

A peculiarity of this junction is its tight coupling with the neighboring one (located in the bottom left of the figure). In the baseline scenarios, their signal plans are synchronized to allow for smooth transit along the major corridor (Krefelder Straße). This means that any significant deviation from the fixed signal plan at one of the junctions (e.g. due to granting absolute priority to the bus) would lead to a disruption of the coordinated signal timings, causing significant delays for both transit and private vehicles at both junctions. Therefore, our PTP strategy needs to be designed in a way that minimizes the impact on the overall signal coordination, while still providing effective prioritization for the bus.

Another important aspect to consider is the presence of a bus stop located directly within the signal approach zone, which introduces additional uncertainty in the bus's ETA at the stopline, as this strategy keeps building upon the dynamic dwell times logic (inclusive of the request stops).

3.3.2. Prioritization logic overview

A general overview of the prioritization logic is shown in **To do: insert figure of the complete diagram**. The logic is composed of two main modules: an Active Prioritization (AP) module, which is responsible for granting priority to the bus when it is approaching the junction, and an Offset Recovery (OR) module, which is responsible for resyncing the traffic light with its master cycle after granting priority to the bus.

The detection algorithm monitors a 200-meter approach zone to detect the presence of the bus (before the bus stop). Once a bus is detected and its dynamic dwell time is calculated and assigned, the algorithm delegates control of the traffic light signal to the Active Prioritization module, which evaluates the bus's ETA at the stopline and the current traffic light state to decide how to manage the signal phases and grant priority to the bus. If no bus is detected, the Offset Recovery module is responsible for resyncing the traffic light with its master cycle, by applying a recovery strategy that systematically compresses the bus phases over the next cycles to safely bring the junction back into coordination. This specific behavior was implemented because early testing showed that green time compensation without re-syncing still lead to spillover and queue build-up for the cross-traffic.

Detection step

The detection step logic is displayed in **To do: insert detection step diagram**. The objective of the detection step is to detect the bus and estimate an arrival time window to facilitate the prioritization operations.

The actual detection is delegated to a “virtual detector” situated approximately 200 meters from the junction's stopline. By virtual detector, we mean that the detection does not rely on a physical induction loop (or similar sensors), but instead simulates the communication between vehicle and infrastructure typical of V2I systems. Thus, the bus (via TraCI) continuously communicates its state to the traffic light controller, allowing the controller to be aware of its speed, position, as well as of its status at the bus stop.

After the controller is made aware of the approximate duration of the dwell time at the stop, it uses an average approaching speed of 8.5 m/s and the dwell time to compute the bus's minimum and maximum travel time windows to the stopline, and then delegates further operations to the AP module.

Active Prioritization Logic

Once the bus's arrival time windows are predicted, the algorithm evaluates the current traffic light state and intervenes dynamically, based on the currently active phase (waiting for a bus or major phase if it currently is in an interstage phase).

3.3.2.0.1. Active phase: major

If the active phase is the opposing major flow, the primary goal is to minimize disruption. The algorithm calculates the minimum time required to safely switch to the bus phase (clearance

and interstage times). If the bus is still far away, the algorithm calculates a delay and artificially extends the major phase. This “Just-In-Time” switching maximizes the green time for private vehicles, allowing cross-traffic queues to dissipate before granting the green light exactly when the bus arrives. Conversely, if the bus is very close, an early green is triggered immediately (if minimum green times are respected). **To do: insert major phase diagram**

3.3.2.0.2. Active phase: bus

If the bus phase is already active, the logic evaluates if the bus will clear the intersection before the maximum green time expires. If not, it decides between two actions:

1. If the bus is far away and there is enough time (e.g., the ETA is greater than the minimum cycle time minus 10 seconds), it inserts a reduced cycle: it drops the bus phase and cycles through the opposing phases, allowing the bus to catch up with the signal plan and then reopening for the bus when it is expected to arrive at the stopline (following the same “Just-In-Time” strategy applied in the major phase). This allows to minimize the disruption for the cross-traffic while still granting priority to the bus.
2. Otherwise, it simply triggers a green extension to hold the phase until the bus clears.

To do: insert bus phase diagram

Offset Recovery

If there is no bus in the detection range, the algorithm then checks for cycle deviation, meaning a drift of the traffic light out of sync with its master cycle due to previous prioritization interventions. If a deviation is detected, the Offset Recovery module is activated to systematically bring the traffic light back in sync with its master cycle.

The drift is computed by continuously tracking the master cycle timeline (ideal timeline) and comparing it to the current active phase and time. The recovery strategy is simple, and is based on a systematic reduction of the green time for the bus phase (up to a predefined maximum reduction of 25 seconds) until the cycle is in-sync with the master cycle. **To do: insert offset recovery diagram**

3.3.3. Results and Performance Evaluation

In the subsequent sections, we analyze the results of the implemented strategies in terms of their impact on average dwell times, travel times and reliability, fuel consumption and emissions. The simulated scenarios are referred to as:

- SQ: baseline scenario (status quo) with fixed dwell times and no prioritization
- DDT: scenario with dynamic dwell times
- RS: scenario with dynamic dwell times and request stops
- PTP: scenario with dynamic dwell times, request stops and public transport prioritization

We will first focus on the impact of the implementation of dynamic dwell times and request stops, and subsequently compare these results with the ones from the PTP scenario.

As stated in Section 3, the results presented in this section represent the aggregate performance of the four simulated scenarios across five runs using random seeds, to isolate the impact of the proposed strategies from the simulator’s intrinsic noise.

Dwell times and travel time

The implementation of dynamic dwell times makes the simulated scenarios more realistic, introducing stochastic elements related to the actual passenger demand. The average dwell times of the DDT, RS and SQ scenarios are shown in **To do: insert avg dwell times**: in the DDT scenario, we have an average dwell time reduction of around 40% compared to the status quo (from 30 seconds to 16-18, depending on the specific bus stop). The implementation of request stops further reduces the average dwell time, especially at the stops with lower passenger activity (as, for example, the bus stop number 4), where the bus can skip the stop entirely.

While dynamic dwell times do lead to a slight reduction of the bus travel time, as seen in **To do: insert travel time distribution plot**, their impact on the line's reliability, quantified as the variability of the travel times, is negligible. The implementation of request stops also does not lead to significant improvements in this aspect.

This suggests that the travel time variance in this corridor is mainly driven by infrastructure-induced delays, such as non-coordinated traffic signal timings and congestion.

Environmental impact

The reduction in dwell and travel times is directly reflected in the environmental performance of the bus line. As shown in **To do: insert fuel plot**, the average fuel consumption per trip decreases from 1.05L in the SQ scenario to 1.0L in the DDT scenario (a 5% reduction), and further to 0.96L in the RS scenario (a 10% reduction compared to SQ).

The emissions of pollutants and green house gases also show a similar, more pronounced downward trend, as shown in **To do: insert emission plot base**:

- Average CO₂ emissions decrease by 5% in the DDT scenario and by 8% in the RS one;
- Average CO emissions decrease by 13% in the DDT scenario and by 18% in the RS one;
- Average NO_x emissions decrease by 14% in the DDT scenario and by 20% in the RS one.

The reduction in pollutants such as CO and NO_x is particularly significant: while CO₂ emissions are almost directly proportional to fuel consumption, CO and NO_x emissions show a higher sensitivity to the actual engine operation. This behavior is simulated by SUMO's default PHEMlight emissions model, which accounts for this by assigning different emission factors to different engine states (e.g., idling, cruising, accelerating). The implementation of DDT leads to a reduction in the idle time of the bus, and the RS system also reduces the number of accelerations to cruise speed after the bus stops, which appear to be the most emission-intensive operational states.

Effectiveness and trade-offs of Public Transport Prioritization

In order to highlight the effectiveness of the PTP strategy in reducing bus delays at the Krefelder Straße junction, we compared the behavior of buses to those in the SQ scenario by means of time-space diagrams, which show the position of the bus along its route over time. The diagrams shown in this section represent the position of buses and personal vehicles as resulted from the first SQ and PTP simulations.

As shown in **To do: insert time-space diagram for sq**, the bus incurs in several delays at the traffic stop (**To do: insert zoom**) due to the non-prioritized traffic light.

Our custom strategy, whose results can be seen in **To do: insert time-space diagram for PTP**, effectively eliminates these delays, as shown in the smooth bus trajectory lines. This highlights the effectiveness of our prioritization strategy.

The benefits to the bus line, however, have to be weighed against the potential disruption to the overall traffic flow at the junction. A first empirical observation showed no apparent spillover effects or congestion caused by the dynamic traffic light logic. A more quantitative analysis (**To do: insert crossing/corridor trt**) of the phenomenon shows an average travel time increase for private vehicles in the crossing stream of roughly 8s compared to the SQ scenarios, while the average travel time for the bus is reduced by around 20s. Private vehicles in the bus corridor also benefit from an average travel time reduction of around 10s.

This trade-off needs to be taken in consideration when giving an overall — of the PTP scenario.

Comparison of active traffic control vs ddt and rs

The implementation of active traffic control via PTP yields significant improvements in bus line performance, effectively addressing the infrastructure-induced limits of the DDT and RS scenarios.

As the active control is implemented on top of the RS scenario, the average dwell times remain consistent (small differences due to the simulator's intrinsic stochasticity), while the travel time distribution **To do: insert travel time distr with ptp** highlights several improvements:

- Distinctly lower mean and median travel times compared with to the other scenarios (15s reduction);
- Consistent reduction in the number of outliers (as shown by the much shorter tails on both ends of the distribution);
- Drastic reduction in distribution variance.

This results in a major improvement to the reliability of the bus line.

The environmental benefits are also significant and show incremental improvements over the previous scenarios **To do: insert fuel / emissions ptp**: average fuel consumption is reduced by 14% compared to SQ at 0.91L per trip, while average pollutant emissions show consistent reductions of 14% with respect to CO₂ and of 25% with respect to CO and NO_x.

While these results are already impressive, we note that this performance gains are achieved by prioritizing a single junction on the bus route. The benefits of an active traffic control strategy could be compounded if implemented across multiple junctions or successive signalized intersections on the bus route course. This would help further reduce absolute travel times while at the same time increasing the reliability of the line and improving its environmental (and economic) performance.

Bibliography

- Cascetta, E., Inaudi, D., & Marquis, G. (1993). Dynamic Estimators of Origin-Destination Matrices Using Traffic Counts. *Transportation Science*, 27(4), 363–373. <https://doi.org/10.1287/trsc.27.4.363>[◦]
- Dios Ortúzar, J. de, & Willumsen, L. G. (2024). *Modelling transport*. John Wiley & sons.
- Flyvbjerg, B., Skamris Holm, M. K., & Buhl, S. L. (2003). How common and how large are cost overruns in transport infrastructure projects?. *Transport Reviews*, 23(1), 71–88. <https://doi.org/10.1080/01441640309904>[◦]
- Flyvbjerg, B., Skamris Holm, M. K., & Buhl, S. L. (2005). How (In)accurate Are Demand Forecasts in Public Works Projects?: The Case of Transportation. *Journal of the American Planning Association*, 71(2), 131–146. <https://doi.org/10.1080/01944360508976688>[◦]
- Hellinga, B. R. (1998). Requirements for the calibration of traffic simulation models. *Proceedings of the Canadian Society for Civil Engineering*, 4, 211–222.
- Lopez, P. A., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flötteröd, Y.-P., Hilbrich, R., Lücken, L., Rummel, J., Wagner, P., & Wiessner, E. (2018). Microscopic Traffic Simulation using SUMO. *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, 2575–2582. <https://doi.org/10.1109/ITSC.2018.8569938>[◦]
- Qurashi, M., Ma, T., Chaniotakis, E., & Antoniou, C. (2020). PC-SPSA: Employing Dimensionality Reduction to Limit SPSA Search Noise in DTA Model Calibration. *IEEE Transactions on Intelligent Transportation Systems*, 21(4), 1635–1645. <https://doi.org/10.1109/TITS.2019.2915273>[◦]
- Spall, J. C. (1998). An overview of the simultaneous perturbation method for efficient optimization. *Johns Hopkins Apl Technical Digest*, 19(4), 482–492.
- Toledo, T., & Koutsopoulos, H. N. (2004). Statistical Validation of Traffic Simulation Models. *Transportation Research Record*, 1876(1), 142–150. <https://doi.org/10.3141/1876-15>[◦]